

Reviewer Pack — Minimal Root Count of Meromorphic Functions and Communicatio...

Entry edb0a853b814 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 20:47:11 UTC

Minimal Root Count of Meromorphic Functions and Communication Complexity Rank Correlation

Entry ID: edb0a853b814 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 20:47:11 UTC

1. Conjecture

Field A (mathematical branch): Complex Analysis **Field B** (complexity object): Communication Complexity

Statement:

For every boolean function f with communication complexity rank r_f , the number of distinct roots of its associated meromorphic function is at most $C \cdot r_f^2$ for some constant C .

Rationale (proposer's reasoning):

Meromorphic functions can encode information in their complex dynamics, potentially reflecting the complexity of computational tasks. The conjecture suggests a connection between the complexity of communication and the algebraic structure of these functions, which has not been explored before in complexity theory.

Taxonomy category: Complex_Analysis × Communication_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 90930c8449cb8355

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A conjecture is supported if, for all generated boolean functions f with communication complexity rank r_f , the ratio of the number of distinct roots to r_f^2 is less than or equal to a constant C within 30 seeds and Pearson correlation coefficient ≥ 0.7 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "meromorphic functions" AND "communication complexity" AND "root count" - "Complex Analysis" AND "Communication Complexity" AND "rank correlation" - "boolean function" AND meromorphic function" AND communication complexity rank"

Top relevant hits considered: - [http://arxiv.org/abs/1112.2313v2] QBF-Based Boolean Function Bi-Decomposition - [http://arxiv.org/abs/2111.08793v3] The Stochastic Boolean Function Evaluation Problem for Symmetric Boolean Functions - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.7s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = len(f)
        if n == 1:
            return 1
        rank = 0
        for i in range(1, n):
            if any(f[j] != f[j ^ (1 << i)] for j in range(2**(n-1))):
                rank += 1
        return rank

    def riemann_roch_theorem(n, r):
        # Simplified version of Riemann-Roch theorem for meromorphic functions
        return n + 1 - r

    def count_distinct_roots(f):
        # Placeholder for counting distinct roots using a simple method
        return len(set(f))

    C = 0.5 # Placeholder constant, adjust as needed
```

```

instances_tested = 0
total_ratio = 0
n_max = 1

for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    if n > n_max:
        n_max = n

    f = generate_random_boolean_function(n)
    r_f = communication_complexity_rank(f)
    roots_count = count_distinct_roots(f)

    if r_f == 0:
        continue

    instances_tested += 1
    ratio = Fraction(roots_count, r_f**2)
    total_ratio += ratio

mean_ratio = total_ratio / instances_tested
conjecture_holds = all(ratio <= C for ratio in [Fraction(roots_count, r_f**2) for _ in range(30)])

return {
    "metric_name": "Ratio of distinct roots to r_f^2",
    "metric_value": mean_ratio,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(range(30, 100))
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4972afbd.py", line 80, in <module>
    result = run_trial(seed)
             ~~~~~^~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4972afbd.py", line 53, in run_trial
    r_f = communication_complexity_rank(f)
          ~~~~~^~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4972afbd.py", line 30, in communication_complexity_rank
    if any(f[j] != f[j ^ (1 << i)] for j in range(2**(n-1))):
        ~~~~~^~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4972afbd.py", line 30, in <genexpr>
    if any(f[j] != f[j ^ (1 << i)] for j in range(2**(n-1))):
        ~~~~~^~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's support conditions. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13360
2	preregistration	ollama_remote	glm4:latest	0	0	12889
3	novelty	ollama_remote	glm4:latest	0	0	12940
4	novelty	ollama_remote	glm4:latest	0	0	15007
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14647
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19660
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16589
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30832
9	judge	ollama_remote	glm4:latest	0	0	22966

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 158891 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/edb0a853b814.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/edb0a853b814.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/edb0a853b814.tar.gz` (if generated)